

# Selección de arquitecturas y herramientas de programación

---

**Desarrollo web en entorno cliente — 2.º DAW**

**Unidad didáctica 1**

# 1. Desarrollo de software

---

## Definición

Proceso sistemático de creación de un programa informático o conjunto de programas para resolver un problema o automatizar una necesidad organizativa.

## Objetivos principales

- **Automatización de procesos:** Sustituir tareas manuales repetitivas por ejecuciones programadas.
- **Aumento de la eficiencia:** Generar ventajas competitivas mediante la optimización de recursos y tiempos.

## Contexto actual en el entorno web

El desarrollo web se ha especializado ampliamente. Ya no se construyen sitios estáticos aislados, sino aplicaciones complejas estructuradas en capas con responsabilidades delimitadas.

# Ciclo de vida del software: fases iniciales

---

## 1. Planificación y análisis de requisitos

- Análisis conducido por perfiles sénior junto con el cliente.
- Estudio de viabilidad técnica, económica y operativa.
- Evaluación y prevención de riesgos.

## 2. Definición de requisitos

- Documentación exhaustiva de lo que la aplicación debe hacer.
- Aprobación formal por parte del cliente antes de comenzar a codificar.

## 3. Diseño de la arquitectura

- Elección de la estructura interna del sistema, definición de módulos y protocolos de comunicación.
- Definición de estándares de calidad y mantenimiento.

# Ciclo de vida del software: fases finales

---

## 4. Construcción (codificación) o implementación

- Escritura de código utilizando lenguajes, entornos de desarrollo, compiladores o intérpretes.

## 5. Pruebas

- Detección, informe, depuración y corrección de errores (*bugs*).
- Verificación del cumplimiento de los requisitos iniciales.

## 6. Despliegue y mantenimiento

- Puesta en marcha de la aplicación en el entorno de producción.
- Corrección de fallos en entorno real y actualización continua del software.

# Modelos de ciclo de vida del software

---

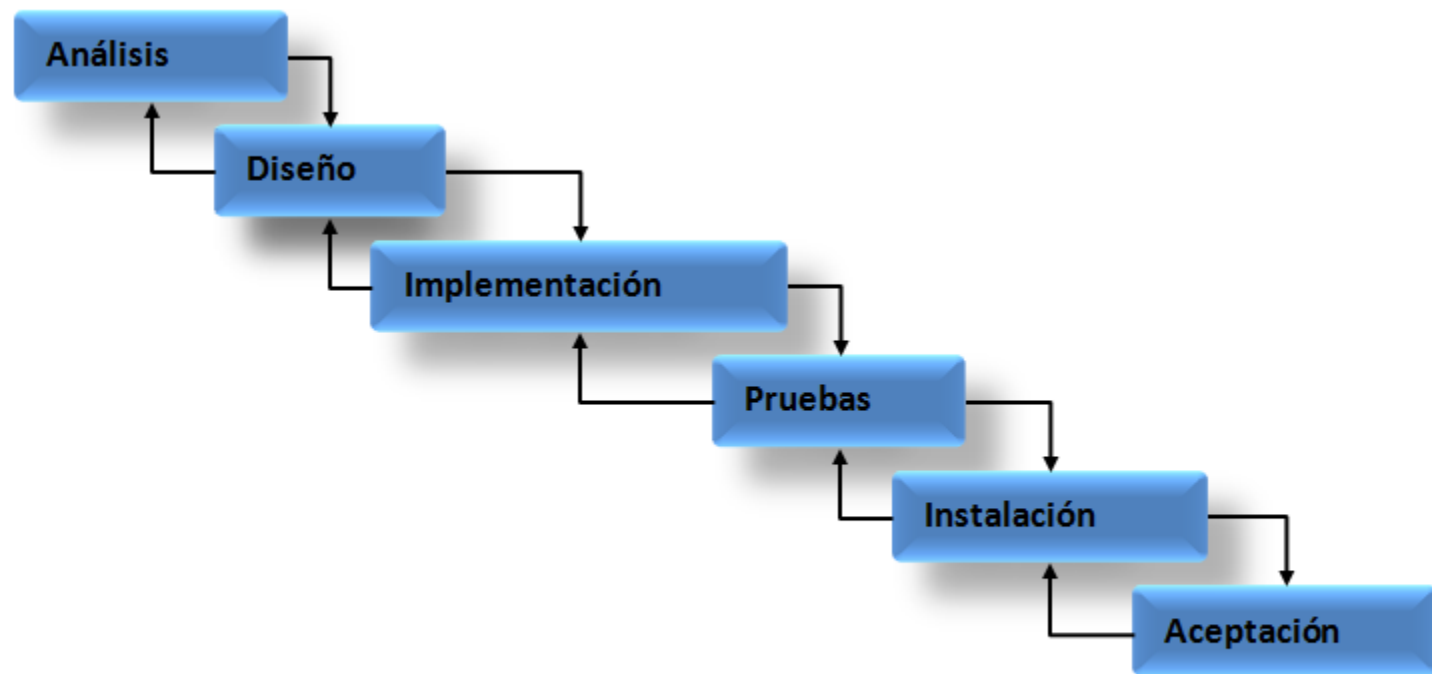
Las fases anteriores pueden organizarse mediante distintas metodologías:

- **Modelo en cascada (*waterfall*):** Secuencial rígido. Cada fase exige la finalización previa de la anterior.
- **Modelo iterativo:** Se repite el ciclo completo en periodos cortos para refinar el producto de forma continua.
- **Modelo incremental:** La aplicación se construye y entrega por bloques funcionales o módulos utilizables.
- **Modelo en espiral:** Combina el enfoque iterativo con una evaluación continua de riesgos en cada vuelta.
- **Modelo de prototipos:** Creación de versiones preliminares sencillas para validar ideas directamente con los usuarios.

## Palabras que ya conocemos ¿sí? :-) --> metodologías y desarrollo

---

- **Waterfall (Cascada):** Término que hace referencia a la forma en que las fases del proyecto avanzan hacia abajo, como una caída de agua, sin posibilidad de volver atrás sin un gran coste.
- **Bug (Error de software):** Literalmente "bicho". Se usa para denominar cualquier fallo, defecto o comportamiento no deseado en el código de un programa.
- **Debugging (Depuración):** Proceso sistemático de buscar, aislar y corregir fallos (*bugs*) dentro del código fuente.
- **Deploy o Despliegue:** Proceso de trasladar el código compilado o interpretado a un servidor de producción para que los usuarios finales puedan utilizarlo.



Ciclo de vida en cascada

## 2. Desarrollo lado cliente vs. lado servidor

---

| Criterio     | Lado cliente ( <i>front-end</i> )                             | Lado servidor ( <i>back-end</i> )                               |
|--------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| Enfoque      | Interfaz visual, comportamiento e interacción con el usuario. | Lógica de negocio, base de datos y administración del servidor. |
| Tecnologías  | HTML, CSS, JavaScript, SASS.                                  | Java, Python, PHP, Ruby, Node.js, C#.                           |
| Ejecución    | Se ejecuta en el navegador del usuario final.                 | Se ejecuta en los servidores de la empresa o nube.              |
| Objetivo     | Accesibilidad, usabilidad y respuesta fluida.                 | Seguridad, eficiencia, escalabilidad y procesamiento de datos.  |
| Herramientas | React, Angular, Vue.js.                                       | Spring Boot, Django, Laravel, Express.                          |

## ***front-end y back-end***

- ***Front-end*** (Desarrollo en el lado del cliente):

- Significa literalmente "extremo frontal".
- Engloba todo lo que el usuario ve, toca e interactúa en el navegador.
- Se encarga de la estructura (HTML), la estética (CSS) y la interactividad en tiempo real (JavaScript).

- ***Back-end*** (Desarrollo en el lado del servidor):

- Significa literalmente "extremo posterior" o "trasero".
- Comprende la infraestructura lógica que no es visible para el usuario.
- Gestiona la autenticación, las reglas de negocio, el acceso a bases de datos y la seguridad del sistema.

## ***framework, UI y UX***

---

- ***Framework*** (Marco de trabajo):

- Estructura de código predefinida que proporciona herramientas, librerías y patrones para desarrollar aplicaciones más rápido y con mayor consistencia. Evita reescribir código común desde cero.

- ***UI*** (*User Interface* / Interfaz de usuario):

- El conjunto de elementos visuales (botones, tipografías, colores, menús, formularios) con los que interactúa el usuario.

- ***UX*** (*User Experience* / Experiencia de usuario):

- La percepción o sensación que tiene el usuario al utilizar la aplicación. Busca que la navegación sea intuitiva, rápida, sin frustraciones y accesible.

## "full stack" y "responsive design"

---

- **Full stack (Desarrollador de pila completa):**
  - Hace referencia a la "pila tecnológica" (*stack*). Un desarrollador *full stack* posee conocimientos y capacidad para trabajar tanto en la capa de *front-end* como en la de *back-end*.
- **Responsive web design (Diseño web adaptativo o como decimos en plan poderno, responsive):**
  - Técnica de maquetación que permite que una página web adapte su estructura y apariencia visual automáticamente según el tamaño de la pantalla del dispositivo (móvil, tableta, ordenador).
- **Single Page Application (SPA):**
  - Aplicación web de página única donde el contenido se carga de forma dinámica sin necesidad de recargar la página entera en el navegador.

### 3. Arquitectura de aplicaciones web: tres niveles

---

La arquitectura de tres niveles (*three-tier architecture*) separa las responsabilidades de una aplicación en tres capas lógicas e independientes:

1. **Nivel de presentación (*presentation tier*):** Es la interfaz visual en el navegador del usuario.
2. **Nivel de aplicación o lógica (*application tier*):** Procesa las reglas del negocio e intermediaria datos.
3. **Nivel de datos (*data tier*):** Almacena y gestiona la información en bases de datos.

#### Beneficio clave

Permite modificar, escalar o sustituir cualquiera de los niveles sin necesidad de reescribir los otros dos.

# Detalle de los niveles de la arquitectura

---

## Nivel de presentación

- Representado en el servidor por el **servidor web**.
- Se ejecuta en el cliente mediante HTML, CSS y JavaScript.

## Nivel de aplicación

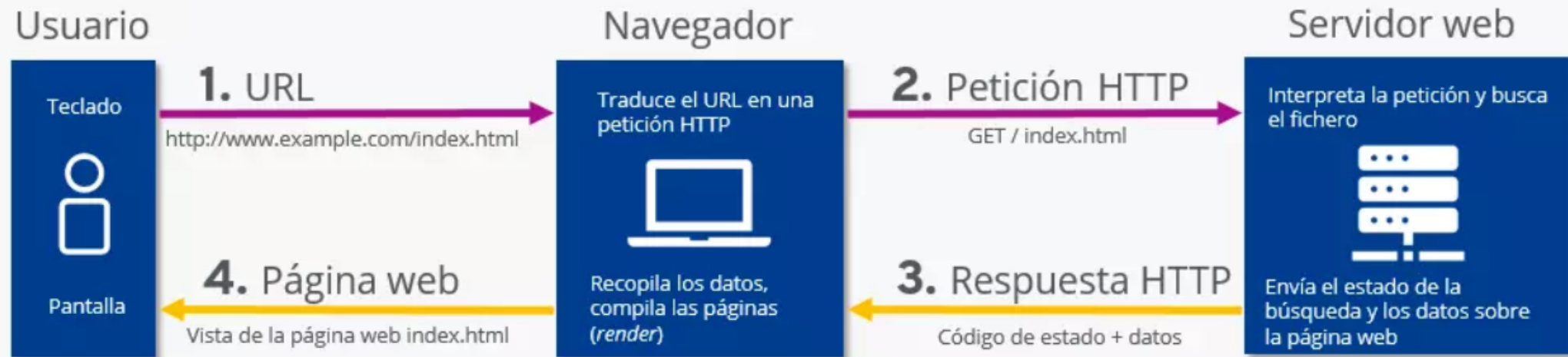
- Representado por el **servidor de aplicaciones**.
- Ejecuta la lógica central del negocio en lenguajes como Java, Python o PHP. Recibe peticiones, realiza cálculos y solicita datos.

## Nivel de datos

- Representado por el **servidor de bases de datos** (relacionales como MySQL/PostgreSQL o NoSQL como MongoDB).
- Guarda de manera persistente la información de la aplicación.

# Flujo de comunicación e interacción HTTP

## El proceso de comunicación según HTTP



## IMPORTANTE

---

**Regla de aislamiento:** El nivel de presentación y el nivel de datos **nunca** deben comunicarse directamente; todo pasa obligatoriamente por el nivel de aplicación.

## 4. Herramientas complementarias: Node.js

---

### Definición

Entorno de ejecución de JavaScript fuera del navegador, basado en el motor V8 de Google Chrome.

### Uso en el lado servidor

- Permite construir aplicaciones web, servidores y API utilizando JavaScript en el *back-end*.

### Utilidad para el desarrollo en el lado cliente

- Permite ejecutar pequeños scripts de JavaScript directamente en la consola sin necesidad de crear un archivo HTML ni abrir el navegador.
- Funciona como base para gestores de paquetes y herramientas modernas de construcción web.

## Herramientas complementarias: Git

---

Sabéis GIT, no ¿no? ¿NO?